



The philosophy of OO programming

Type	Lecture
Date	@January 8, 2022
Lecture #	1
Lecture URL	https://youtu.be/PF6bAB-d9hl
Notion URL	https://21f1003586.notion.site/The-philosophy-of-OO-programming-43af67bf48ea4fd99b7b3a7a488fb8f3
Week #	3

Algorithms + Data Structures = Programs

- Title of Niklaus Wirth’s introduction to Pascal
- Traditionally, algorithms come first
- Data representation comes later

Object Oriented design

- Reverses the traditional focus
- First, identify the data we want to maintain and manipulate
- Then, identify algorithms to operate on the data
- **Claim:** Works better for large systems
- **Example:** Simple web browser
 - 2000 procedures maintaining global data
 - ... vs. 100 classes, each with about 20 methods
 - Much easier to grasp the design
 - Debugging: an object is in an incorrect state
 - Search among 20 methods rather than 2000 procedures

Object Oriented design: Example

- An order processing system typically involves

- Items
- Orders
- Shipping addresses
- Payments
- Accounts
- What happens to these objects?
 - Items are *added* to orders
 - Orders are *shipped, cancelled*
 - Payments are *accepted, rejected*
- Nouns signify objects, verbs denote the methods that operate on objects
 - Associate with each order, a method to add an item

Designing objects

- Behaviour — what methods do we need to operate on objects?
- State — how does the object react when methods are invoked?
 - State is the info in the instance variables
- Encapsulation — should not change unless a method operates on it
- Identity — distinguish between different objects of the same class
 - State may be the same — two orders may contain the same item
- These features interact
 - State will typically affect behaviour
 - Cannot add an item to an order that has been shipped
 - Cannot ship an empty order

Relationship between classes

- Dependence
 - `Order` needs `Account` to check credit status
 - `Item` does not depend on `Account`
 - Robust design minimizes dependencies, or coupling between classes
- Aggregation
 - `Order` contains `Item` objects
- Inheritance
 - One object is a specialized version of another
 - `ExpressOrder` inherits from `Order`
 - Extra methods to compute shipping charges, priority handling

Summary

- An object-oriented approach can help organize code in large projects